Physics-Informed Neural Networks HW SciML

In [1]:
```
import os, time, random
import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

DEVICE = torch.device("cpu")      # problems are tiny; CPU is as fast as GPU here
torch.set_default_dtype(torch.float32)
FIG_DIR = "figures"
os.makedirs(FIG_DIR, exist_ok=True)


def set_seed(seed: int = 0) -> None:
    """Seed every random number generator so runs are reproducible."""
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)


class MLP(nn.Module):
    """Fully connected network: in_dim -> [hidden, tanh] x n_layers -> out_dim (linear output)."""

    def __init__(self, in_dim: int, out_dim: int, hidden: int, n_layers: int):
        super().__init__()
        layers = [nn.Linear(in_dim, hidden), nn.Tanh()]
        for _ in range(n_layers - 1):
            layers += [nn.Linear(hidden, hidden), nn.Tanh()]
        layers.append(nn.Linear(hidden, out_dim))
        self.net = nn.Sequential(*layers)
        for m in self.net:                        # Xavier init keeps tanh out of saturation
            if isinstance(m, nn.Linear):
                nn.init.xavier_normal_(m.weight)
                nn.init.zeros_(m.bias)

    def forward(self, x):
        return self.net(x)


def grad(y, x):
    """Point-wise dy/dx by autograd. create_graph=True allows higher derivatives and backprop."""
    return torch.autograd.grad(y, x, grad_outputs=torch.ones_like(y), create_graph=True)[0]


def relative_l2(pred, exact):
    """Relative L2 error ||pred - exact|| / ||exact||."""
```

```
    return float(np.linalg.norm(pred - exact) / np.linalg.norm(exact))


class BestCheckpoint:
    """Copy of the weights with the lowest unweighted training loss.
Selecting by training loss
    never looks at the analytical solution, so it is a legitimate model-
selection rule."""

    def __init__(self):
        self.epoch, self.loss, self.state = 0, float("inf"), None

    def update(self, epoch, loss, model):
        if not loss < self.loss:                # also rejects NaN
            return
        self.epoch, self.loss = epoch, loss
        self.state = {k: v.detach().clone() for k, v in
model.state_dict().items()}

    def store(self, hist):
        hist["best_epoch"], hist["best_loss"], hist["best_state"] =
self.epoch, self.loss, self.state


def to_numpy(t):
    return t.detach().cpu().numpy().ravel()


def n_params(model):
    return sum(p.numel() for p in model.parameters())
```

Problem 1: 1D Steady-State Heat Equation

$$\frac{d^2T}{dx^2} + \frac{q(x)}{\kappa} = 0, \quad x \in [0,1], \qquad T(0) = T(1) = 0, \quad \kappa = 0.5, \quad q(x) = 15x - 2$$

Part a: Network design

The network $T_\theta(x)$ is a multilayer perceptron with architecture $1 \to 32 \to 32 \to 32 \to 1$. Each hidden layer applies a linear map followed by a hyperbolic tangent activation, and the output layer is linear so the predicted temperature is unbounded. Weights use Xavier normal initialization and biases start at zero.

The tanh activation is chosen because it is infinitely differentiable. The physics loss requires T'', and an activation such as ReLU has a second derivative of zero almost everywhere, which would make the PDE residual uninformative.

In [2]:
```python
KAPPA = 0.5

def q(x):
    """Heat source q(x) = 15x - 2 (works for torch and numpy)."""
    return 15.0 * x - 2.0

def T_exact(x):
    """Analytical solution, derived in Part e."""
    return -5.0 * x**3 + 2.0 * x**2 + 3.0 * x

def make_model():
    return MLP(in_dim=1, out_dim=1, hidden=32, n_layers=3).to(DEVICE)

def best_model(hist):
    """Network rebuilt with the lowest-loss weights recorded during
training."""
    if hist.get("best_state") is None:
        raise ValueError("history has no best checkpoint")
    model = make_model()
    model.load_state_dict(hist["best_state"])
    return model

print(make_model())
print("Trainable parameters:", n_params(make_model()))
```

```
MLP(
  (net): Sequential(
    (0): Linear(in_features=1, out_features=32, bias=True)
    (1): Tanh()
    (2): Linear(in_features=32, out_features=32, bias=True)
    (3): Tanh()
    (4): Linear(in_features=32, out_features=32, bias=True)
    (5): Tanh()
    (6): Linear(in_features=32, out_features=1, bias=True)
  )
)
Trainable parameters: 2209
```

Part b: Physics loss

At each collocation point x_i the PDE residual and the physics loss are

$$r(x_i) = \frac{d^2T_\theta}{dx^2}(x_i) + \frac{q(x_i)}{\kappa}, \qquad \mathcal{L}_{Physics} = \frac{1}{N_f}\sum_{i=1}^{N_f} r(x_i)^2$$

Both derivatives are computed exactly with automatic differentiation. The second derivative is obtained by differentiating the first, and the graph is kept so the loss can be back-propagated to the weights.

Part c: Boundary loss

$$\mathcal{L}_{BCs} = \frac{1}{2}\left[T_\theta(0)^2 + T_\theta(1)^2\right]$$

Because the Dirichlet targets are zero, the boundary error is the predicted temperature at each end of the rod.

In [3]:
```python
def collocation_points(n_f: int = 100):
    """N_f equidistant points on [0, 1] with gradient tracking (needed for dT/dx)."""
    return torch.linspace(0.0, 1.0, n_f, device=DEVICE).reshape(-1, 1).requires_grad_(True)

def boundary_points():
    return torch.tensor([[0.0], [1.0]], device=DEVICE)

def physics_loss(model, x_f):
    T = model(x_f)
    T_x = grad(T, x_f)                      # dT/dx
    T_xx = grad(T_x, x_f)                   # d2T/dx2
    residual = T_xx + q(x_f) / KAPPA
    return torch.mean(residual**2)

def bc_loss(model, x_b):
    return torch.mean(model(x_b)**2)        # targets are 0
```

Part d: Training

$N_f = 100$ equidistant collocation points (including the endpoints), total loss $\mathcal{L} = \mathcal{L}_{BCs} + \mathcal{L}_{Physics}$, Adam with learning rate 10^{-3}, 10,000 full-batch epochs. The trained model is kept and reused as the fixed-weight baseline in Problems 2 and 3, so it is trained only once.

In [4]:
```python
def train_strong(epochs=10_000, lr=1e-3, n_f=100, seed=0, log_every=1000):
    set_seed(seed)
    model = make_model()
    x_f, x_b = collocation_points(n_f), boundary_points()
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    hist = {"total": [], "bc": [], "phys": []}
    best = BestCheckpoint()
    t0 = time.perf_counter()
    for epoch in range(1, epochs + 1):
        opt.zero_grad(set_to_none=True)
        l_bc, l_phys = bc_loss(model, x_b), physics_loss(model, x_f)
        loss = l_bc + l_phys
        best.update(epoch, loss.item(), model)  # before opt.step(): the loss belongs to the current weights
        loss.backward()
        opt.step()
        hist["total"].append(loss.item()); hist["bc"].append(l_bc.item()); hist["phys"].append(l_phys.item())
        if epoch % log_every == 0 or epoch == 1:
            print(f"[strong] epoch {epoch:6d} | L={loss.item():.3e} | L_BC={l_bc.item():.3e} | L_phys={l_phys.item():.3e}")
```

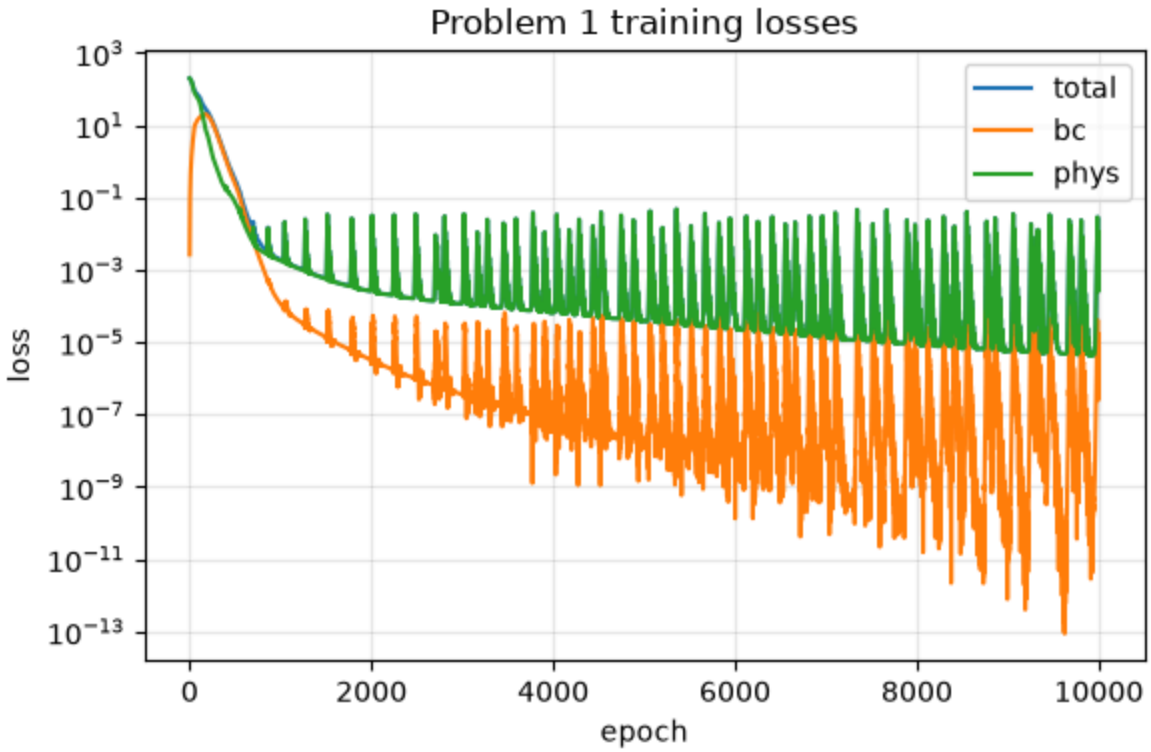
```
    best.store(hist)
    return model, hist, time.perf_counter() - t0


m_strong, h_strong, t_strong = train_strong()
print(f"\nTraining time: {t_strong:.1f} s")
print(f"Lowest training loss: {h_strong['best_loss']:.3e} at epoch
{h_strong['best_epoch']}")

fig, ax = plt.subplots(figsize=(6, 4))
for k in ("total", "bc", "phys"):
    ax.semilogy(h_strong[k], label=k)
ax.set_xlabel("epoch"); ax.set_ylabel("loss"); ax.set_title("Problem 1
training losses")
ax.legend(); ax.grid(alpha=0.3, which="both")
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/p1_losses.png", dpi=150);
plt.show()
```

```
[strong] epoch      1 | L=1.978e+02 | L_BC=2.655e-03 | L_phys=1.978e+02
[strong] epoch   1000 | L=1.902e-03 | L_BC=1.144e-04 | L_phys=1.788e-03
[strong] epoch   2000 | L=2.261e-03 | L_BC=2.596e-06 | L_phys=2.258e-03
[strong] epoch   3000 | L=1.360e-04 | L_BC=3.521e-07 | L_phys=1.356e-04
[strong] epoch   4000 | L=6.577e-05 | L_BC=8.365e-08 | L_phys=6.569e-05
[strong] epoch   5000 | L=3.828e-05 | L_BC=1.697e-08 | L_phys=3.827e-05
[strong] epoch   6000 | L=4.037e-05 | L_BC=8.842e-08 | L_phys=4.028e-05
[strong] epoch   7000 | L=3.451e-04 | L_BC=2.297e-07 | L_phys=3.449e-04
[strong] epoch   8000 | L=1.307e-03 | L_BC=1.311e-06 | L_phys=1.305e-03
[strong] epoch   9000 | L=5.654e-06 | L_BC=9.774e-11 | L_phys=5.654e-06
[strong] epoch  10000 | L=4.559e-03 | L_BC=6.843e-06 | L_phys=4.552e-03

Training time: 18.2 s
Lowest training loss: 4.278e-06 at epoch 9929
```



Training behaviour. The physics term dominates the initial loss by five orders of magnitude ($\mathcal{L}_{Physics} \approx 2 \times 10^{2}$ against $\mathcal{L}_{BCs} \approx 3 \times 10^{-3}$), so early training is driven almost entirely by the PDE residual. The baseline of $\mathcal{L}_{Physics}$ falls steadily to about 5×10^{-6}, but the curve shows regular spikes of two to four orders of magnitude throughout training. These spikes are typical of Adam at a fixed learning rate on a sharp loss surface: the optimizer overshoots, the loss jumps, and it then recovers. The final epoch fell on one of these spikes (total loss 4.6×10^{-3} at epoch 10,000 against 5.7×10^{-6} at epoch 9,000), so the final model is less accurate than the best model seen during training.

Part e: Analytical solution and comparison

Rearranging the PDE: $T'' = -q/\kappa = -(15x - 2)/0.5 = -30x + 4$. Integrating twice,

$$T'(x) = -15x^2 + 4x + C_1, \qquad T(x) = -5x^3 + 2x^2 + C_1 x + C_2$$

$T(0) = 0$ gives $C_2 = 0$, and $T(1) = -5 + 2 + C_1 = 0$ gives $C_1 = 3$:

$$\boxed{T(x) = -5x^3 + 2x^2 + 3x}$$

Check: $T''(x) = -30x + 4 = -q(x)/\kappa$ and $T(0) = T(1) = 0$.

The relative L^2 error is evaluated on a 1,001-point grid, ten times finer than the training grid, so it also measures accuracy between collocation points:

$$\varepsilon = \frac{\|T_{PINN} - T_{exact}\|_2}{\|T_{exact}\|_2}$$

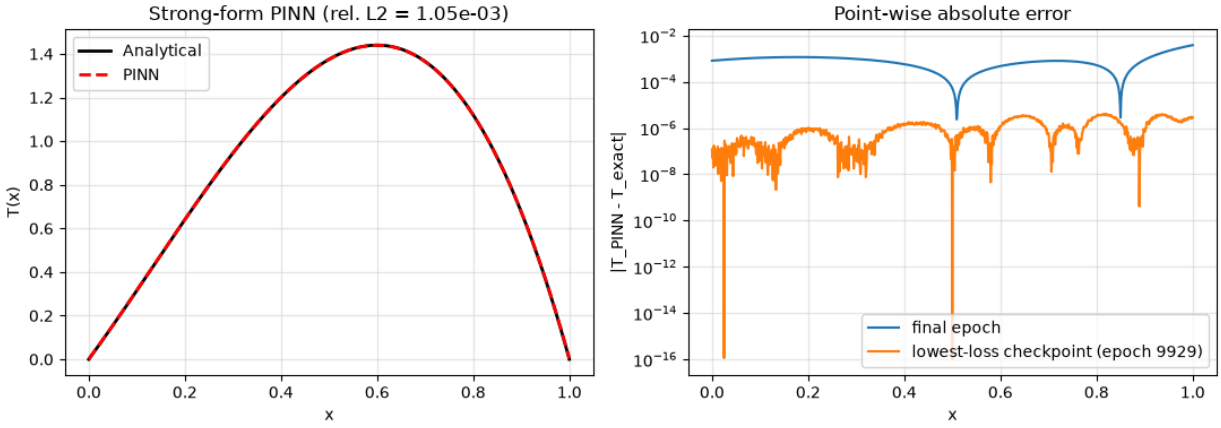
In [5]:
```
def evaluate(model, n_test=1001):
    x_test = np.linspace(0.0, 1.0, n_test)
    with torch.no_grad():
        x_t = torch.tensor(x_test, device=DEVICE,
dtype=torch.get_default_dtype()).reshape(-1, 1)
        T_pred = to_numpy(model(x_t))
    T_true = T_exact(x_test)
    return x_test, T_pred, T_true, relative_l2(T_pred, T_true)


x_test, T_s, T_true, err_strong = evaluate(m_strong)
_, T_s_best, _, err_strong_best = evaluate(best_model(h_strong))
print(f"Relative L2 error (strong-form PINN, final epoch):
{err_strong:.3e}")
print(f"Relative L2 error (lowest-loss checkpoint, epoch
{h_strong['best_epoch']}): {err_strong_best:.3e}")

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(x_test, T_true, "k-", lw=2, label="Analytical")
ax[0].plot(x_test, T_s, "r--", lw=2, label="PINN")
ax[0].set_xlabel("x"); ax[0].set_ylabel("T(x)"); ax[0].legend()
```

```
ax[0].set_title(f"Strong-form PINN (rel. L2 = {err_strong:.2e})")
ax[1].semilogy(x_test, np.abs(T_s - T_true) + 1e-16, label="final epoch")
ax[1].semilogy(x_test, np.abs(T_s_best - T_true) + 1e-16, label=f"lowest-
loss checkpoint (epoch {h_strong['best_epoch']})")
ax[1].legend()
ax[1].set_xlabel("x"); ax[1].set_ylabel("|T_PINN - T_exact|");
ax[1].set_title("Point-wise absolute error")
for a in ax: a.grid(alpha=0.3, which="both")
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/p1_solution.png", dpi=150);
plt.show()
```

```
Relative L2 error (strong-form PINN, final epoch): 1.047e-03
Relative L2 error (lowest-loss checkpoint, epoch 9929): 1.544e-06
```



Discussion. The relative L^2 error is 1.05×10^{-3}. The PINN and analytical curves are indistinguishable at plot scale, including the peak near $x = 0.6$. The point-wise error stays near 10^{-3} across most of the domain and is largest at $x = 1$, where it reaches about 4×10^{-3}. This matches the final boundary loss: $\mathcal{L}_{BCs} = 6.8 \times 10^{-6}$ implies a boundary error of roughly $\sqrt{2 \times 6.8 \times 10^{-6}} \approx 3.7 \times 10^{-3}$. The remaining error is therefore mostly a boundary-condition error left by the last Adam spike rather than an interior PDE error.

Keeping the lowest-training-loss checkpoint confirms this. At epoch 9,929 the training loss was 4.3×10^{-6}, and that checkpoint has a relative L^2 error of 1.5×10^{-6}, almost 700 times lower than the final epoch. Choosing the checkpoint by training loss never uses the analytical solution, so it is a legitimate model-selection rule. Problems 2 and 3 report both the final-epoch and the lowest-loss-checkpoint errors. A decaying learning rate or a final L-BFGS stage would likely remove the spikes altogether.

Problem 2: Loss Function Design

Part a: Adaptive loss weighting

$$\mathcal{L}(t) = \alpha(t)\,\mathcal{L}_{BCs} + \beta(t)\,\mathcal{L}_{Physics}$$

At initialization $\mathcal{L}_{Physics}$ is about 10^5 times larger than $\mathcal{L}_{BCs}$ (Problem 1), so the gradient of the total loss is dominated by the physics term and the boundary conditions are learned more slowly. Adaptive schemes rescale the two terms during training so that both contribute comparably to each update. Two gradient-based schemes are compared with the fixed baseline ($\alpha = \beta = 1$).

Closed-form gradient-norm balancing ("grad. balance", both weights dynamic). With $g_b = \|\nabla_\theta \mathcal{L}_{BCs}\|$ and $g_p = \|\nabla_\theta \mathcal{L}_{Physics}\|$,

$$\hat{\alpha} = \frac{2 g_p}{g_b + g_p}, \qquad \hat{\beta} = \frac{2 g_b}{g_b + g_p}$$

This makes $\hat{\alpha} g_b = \hat{\beta} g_p$, so both terms push on the parameters with equal strength. The constraint $\hat{\alpha} + \hat{\beta} = 2$ fixes the *balance* between the terms, not the step size: Adam divides each update by a running estimate of the gradient magnitude, so it is roughly invariant to the overall gradient scale, and only the ratio α/β matters. This rule is not GradNorm (Chen et al., 2018), which learns the weights through an additional loss.

Learning-rate annealing (Wang, Teng and Perdikaris, 2021). $\beta = 1$ and

$$\hat{\alpha} = \frac{\max_\theta |\nabla_\theta \mathcal{L}_{Physics}|}{\overline{|\nabla_\theta \mathcal{L}_{BCs}|}}$$

It is run twice: as published, and with the estimate capped, $\hat{\alpha} \leftarrow \min(\hat{\alpha}, \alpha_{max})$ with $\alpha_{max} = 10^3$. The reason for the cap is explained in the discussion; caps of 10^2 and 10^4 are also checked.

In all schemes the raw estimates are smoothed with an exponential moving average, $\alpha \leftarrow (1 - \eta)\alpha + \eta\hat{\alpha}$ with $\eta = 0.1$. The gradient of each loss term is computed once per step and the weighted sum is written directly into the parameter gradients, so the adaptive schemes need no extra forward or backward passes.

Implementation note. The output-layer bias does not appear in $\mathcal{L}_{Physics}$, because a constant offset has zero second derivative. Autograd therefore returns no gradient for it, and `term_grads` substitutes zeros (`allow_unused=True` plus an explicit `torch.zeros_like`, which works on any PyTorch 2.x).

Model selection. Every run also keeps its lowest-training-loss checkpoint, chosen by the *unweighted* loss $\mathcal{L}_{BCs} + \mathcal{L}_{Physics}$ so that all schemes use the same criterion as the fixed run.

In [6]:
```
ALPHA_MAX = 1e3        # cap for the Wang scheme; 1e2 and 1e4 are checked
below


def flat_norm(grads):
    return torch.sqrt(sum((g**2).sum() for g in grads))
```

```
def term_grads(loss, params, retain_graph=False):
    """d(loss)/d(params), with zeros for parameters the loss does not
depend on (the output bias in L_Physics)."""
    grads = torch.autograd.grad(loss, params, retain_graph=retain_graph,
allow_unused=True)
    return [torch.zeros_like(p) if g is None else g for p, g in zip(params,
grads)]


def train_adaptive(scheme="gradbal", epochs=10_000, lr=1e-3, n_f=100,
eta=0.1, alpha_max=None, seed=0,
                   log_every=1000):
    if scheme not in ("gradbal", "wang"):
        raise ValueError(f"unknown scheme {scheme!r}")
    set_seed(seed)                                          # same initial
weights as the fixed run
    model = make_model()
    x_f, x_b = collocation_points(n_f), boundary_points()
    params = list(model.parameters())
    opt = torch.optim.Adam(params, lr=lr)
    tag = scheme if alpha_max is None else f"{scheme}, cap {alpha_max:g}"
    alpha, beta = 1.0, 1.0
    hist = {"bc": [], "phys": [], "alpha": [], "beta": []}
    best = BestCheckpoint()                                 # selected on the
unweighted loss, as for the fixed run
    t0 = time.perf_counter()
    for epoch in range(1, epochs + 1):
        l_bc, l_phys = bc_loss(model, x_b), physics_loss(model, x_f)
        best.update(epoch, l_bc.item() + l_phys.item(), model)
        g_bc = term_grads(l_bc, params, retain_graph=True)
        g_ph = term_grads(l_phys, params)
        with torch.no_grad():
            if scheme == "gradbal":
                nb, npy = flat_norm(g_bc), flat_norm(g_ph)
                a_hat = (2 * npy / (nb + npy + 1e-12)).item()
                b_hat = (2 * nb / (nb + npy + 1e-12)).item()
            else:
                max_ph = max(g.abs().max() for g in g_ph)
                mean_bc = torch.cat([g.abs().flatten() for g in
g_bc]).mean()
                a_hat, b_hat = (max_ph / (mean_bc + 1e-12)).item(), 1.0
                if alpha_max is not None:
                    a_hat = min(a_hat, alpha_max)           # breaks the alpha
-> infinity feedback loop
            alpha = (1 - eta) * alpha + eta * a_hat         # EMA smoothing
            beta = (1 - eta) * beta + eta * b_hat
            for p, gb, gp in zip(params, g_bc, g_ph):
                p.grad = alpha * gb + beta * gp             # gradient of
alpha*L_BC + beta*L_phys
        opt.step()
        hist["bc"].append(l_bc.item()); hist["phys"].append(l_phys.item())
        hist["alpha"].append(alpha); hist["beta"].append(beta)
        if epoch % log_every == 0 or epoch == 1:
            print(f"[{tag}] epoch {epoch:6d} | L_BC={l_bc.item():.3e} |
```

```
L_phys={l_phys.item():.3e} "
                  f"| alpha={alpha:.3g} | beta={beta:.3g}")
    best.store(hist)
    return model, hist, time.perf_counter() - t0
```

In [7]:
```
runs = {"fixed": (m_strong, h_strong, t_strong)}          # reuse the
Problem 1 model as the baseline
runs["grad. balance"] = train_adaptive("gradbal")
runs["Wang"] = train_adaptive("wang")
runs["Wang, capped"] = train_adaptive("wang", alpha_max=ALPHA_MAX)

results = {}
print(f"\n{'scheme':14s} {'rel. L2':>10s} {'best epoch':>11s} {'best rel.
L2':>13s} {'final L_BC':>11s} "
      f"{'final L_phys':>13s} {'max alpha':>10s} {'time [s]':>9s}")
for name, (m, h, t) in runs.items():
    x, Tp, Tt, err = evaluate(m)
    err_best = evaluate(best_model(h))[3]
    results[name] = (x, Tp, Tt, err)
    print(f"{name:14s} {err:10.3e} {h['best_epoch']:11d} {err_best:13.3e}
{h['bc'][-1]:11.3e} "
          f"{h['phys'][-1]:13.3e} {max(h.get('alpha', [1.0])):10.2e}
{t:9.1f}")
```

```
[gradbal] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 | alpha=1.1 |
beta=0.901
[gradbal] epoch   1000 | L_BC=6.617e-14 | L_phys=9.404e-01 | alpha=2 |
beta=3.76e-07
[gradbal] epoch   2000 | L_BC=6.439e-15 | L_phys=9.404e-01 | alpha=2 |
beta=7.83e-08
[gradbal] epoch   3000 | L_BC=5.196e-14 | L_phys=9.399e-01 | alpha=2 |
beta=3.46e-07
[gradbal] epoch   4000 | L_BC=8.465e-13 | L_phys=9.361e-01 | alpha=2 |
beta=6.42e-07
[gradbal] epoch   5000 | L_BC=3.563e-10 | L_phys=1.818e-03 | alpha=1.99 |
beta=0.00823
[gradbal] epoch   6000 | L_BC=4.766e-13 | L_phys=1.646e-03 | alpha=2 |
beta=0.000345
[gradbal] epoch   7000 | L_BC=7.550e-14 | L_phys=1.126e-03 | alpha=2 |
beta=0.000136
[gradbal] epoch   8000 | L_BC=8.967e-13 | L_phys=1.095e-03 | alpha=2 |
beta=0.000408
[gradbal] epoch   9000 | L_BC=1.728e-13 | L_phys=6.062e-04 | alpha=2 |
beta=0.000351
[gradbal] epoch  10000 | L_BC=1.821e-12 | L_phys=5.788e-04 | alpha=2 |
beta=0.00138
[wang] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 | alpha=345 | beta=1
[wang] epoch   1000 | L_BC=1.418e-09 | L_phys=1.947e+02 | alpha=1.8e+08 |
beta=1
[wang] epoch   2000 | L_BC=4.081e-05 | L_phys=1.955e+02 | alpha=1.2e+05 |
beta=1
[wang] epoch   3000 | L_BC=1.649e-06 | L_phys=1.956e+02 | alpha=5.2e+05 |
beta=1
[wang] epoch   4000 | L_BC=6.495e-06 | L_phys=1.964e+02 | alpha=1.34e+05 |
beta=1
[wang] epoch   5000 | L_BC=2.978e-06 | L_phys=1.964e+02 | alpha=1.97e+05 |
```

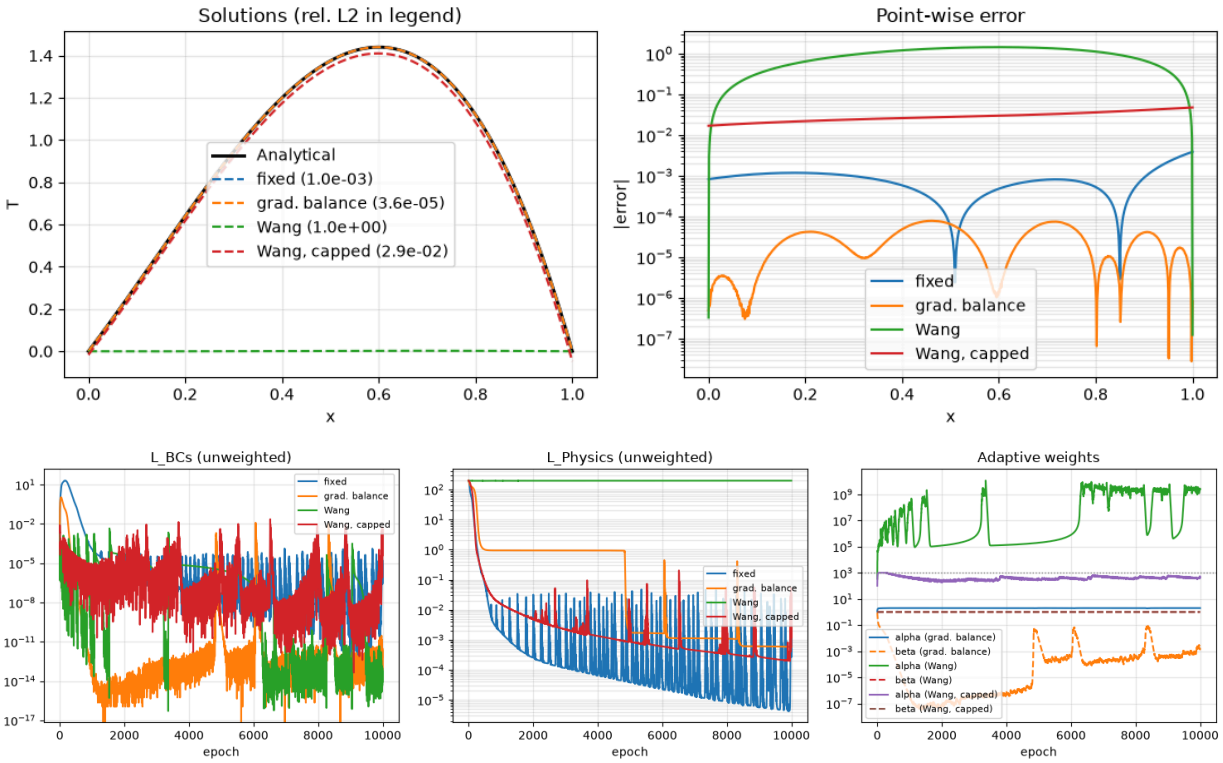
```
beta=1
[wang] epoch   6000 | L_BC=1.817e-07 | L_phys=1.963e+02 | alpha=7.91e+05 |
beta=1
[wang] epoch   7000 | L_BC=1.227e-12 | L_phys=1.963e+02 | alpha=1.91e+09 |
beta=1
[wang] epoch   8000 | L_BC=2.206e-14 | L_phys=1.963e+02 | alpha=2.21e+09 |
beta=1
[wang] epoch   9000 | L_BC=1.657e-14 | L_phys=1.963e+02 | alpha=2.52e+09 |
beta=1
[wang] epoch  10000 | L_BC=2.412e-14 | L_phys=1.967e+02 | alpha=2.46e+09 |
beta=1
[wang, cap 1000] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 | alpha=101
| beta=1
[wang, cap 1000] epoch   1000 | L_BC=4.792e-06 | L_phys=1.559e-02 | alpha=427
| beta=1
[wang, cap 1000] epoch   2000 | L_BC=1.169e-05 | L_phys=4.880e-03 | alpha=196
| beta=1
[wang, cap 1000] epoch   3000 | L_BC=5.356e-10 | L_phys=2.210e-03 | alpha=352
| beta=1
[wang, cap 1000] epoch   4000 | L_BC=3.688e-09 | L_phys=1.342e-03 | alpha=460
| beta=1
[wang, cap 1000] epoch   5000 | L_BC=3.467e-04 | L_phys=1.739e-03 | alpha=369
| beta=1
[wang, cap 1000] epoch   6000 | L_BC=8.281e-08 | L_phys=6.028e-04 | alpha=373
| beta=1
[wang, cap 1000] epoch   7000 | L_BC=5.539e-10 | L_phys=4.506e-04 | alpha=546
| beta=1
[wang, cap 1000] epoch   8000 | L_BC=3.006e-05 | L_phys=7.739e-04 | alpha=431
| beta=1
[wang, cap 1000] epoch   9000 | L_BC=6.392e-09 | L_phys=2.780e-04 | alpha=506
| beta=1
[wang, cap 1000] epoch  10000 | L_BC=6.733e-05 | L_phys=5.981e-04 | alpha=408
| beta=1

scheme            rel. L2  best epoch  best rel. L2  final L_BC  final L_phys
max alpha  time [s]
fixed           1.047e-03        9929     1.544e-06   6.843e-06     4.552e-03
1.00e+00      18.2
grad. balance   3.641e-05       10000     3.540e-05   1.821e-12     5.788e-04
2.00e+00      21.1
Wang            9.988e-01         559     9.492e-01   2.412e-14     1.967e+02
1.13e+10      19.3
Wang, capped    2.943e-02        9957     7.920e-05   6.733e-05     5.981e-04
1.00e+03      18.8
```

In [8]:

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(x_test, T_true, "k-", lw=2, label="Analytical")
for name, (x, Tp, Tt, err) in results.items():
    ax[0].plot(x, Tp, "--", label=f"{name} ({err:.1e})")
    ax[1].semilogy(x, np.abs(Tp - Tt) + 1e-16, label=name)
ax[0].set_title("Solutions (rel. L2 in legend)"); ax[0].set_ylabel("T")
ax[1].set_title("Point-wise error"); ax[1].set_ylabel("|error|")
for a in ax: a.set_xlabel("x"); a.legend(); a.grid(alpha=0.3, which="both")
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/p2_solutions.png", dpi=150);
plt.show()
```

```
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
for name, (m, h, t) in runs.items():
    ax[0].semilogy(h["bc"], label=name)
    ax[1].semilogy(h["phys"], label=name)
for name in ("grad. balance", "Wang", "Wang, capped"):
    ax[2].semilogy(runs[name][1]["alpha"], label=f"alpha ({name})")
    ax[2].semilogy(runs[name][1]["beta"], "--", label=f"beta ({name})")
ax[2].axhline(ALPHA_MAX, color="gray", ls=":", lw=1)
ax[0].set_title("L_BCs (unweighted)"); ax[1].set_title("L_Physics
(unweighted)"); ax[2].set_title("Adaptive weights")
for a in ax: a.set_xlabel("epoch"); a.legend(fontsize=8); a.grid(alpha=0.3,
which="both")
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/p2_losses_weights.png",
dpi=150); plt.show()
```



In [9]:

```
QUIET = 10**9
SEEDS = (0, 1, 2)


def stalled_epochs(h, threshold=0.5, after=500):
    """Epochs after warm-up during which L_Physics is still above threshold
(the plateau in the figure)."""
    return int(np.sum(np.array(h["phys"][after:]) > threshold))


def summarize(m, h):
    return evaluate(m)[3], evaluate(best_model(h))[3], stalled_epochs(h)


configs = {"fixed": None, "grad. balance": dict(scheme="gradbal"), "Wang":
dict(scheme="wang"),
           "Wang, capped": dict(scheme="wang", alpha_max=ALPHA_MAX)}
seed_err = {name: {0: summarize(*runs[name][:2])} for name in configs}
```

```
for s in SEEDS[1:]:
    for name, kw in configs.items():
        m, h, _ = train_strong(seed=s, log_every=QUIET) if kw is None else
train_adaptive(seed=s, log_every=QUIET, **kw)
        seed_err[name][s] = summarize(m, h)

print(f"\n{'scheme':14s} {'seed':>4s} {'final rel. L2':>14s} {'best rel.
L2':>13s} {'stalled epochs':>15s}")
for name, per_seed in seed_err.items():
    for s, (ef, eb, st) in per_seed.items():
        print(f"{name:14s} {s:4d} {ef:14.3e} {eb:13.3e} {st:15d}")

print("\nCap sensitivity (seed 0):")
for cap in (1e2, ALPHA_MAX, 1e4):
    m, h, _ = runs["Wang, capped"] if cap == ALPHA_MAX else
train_adaptive("wang", alpha_max=cap, log_every=QUIET)
    ef, eb, st = summarize(m, h)
    print(f"  alpha_max = {cap:7.0e}: final rel. L2 = {ef:.3e}, best rel.
L2 = {eb:.3e}")


def ms_per_epoch(trainers, epochs=500, repeats=3):
    """Interleaved short runs, best of `repeats`: wall-clock time on a
laptop is noisy, the minimum is the stable statistic."""
    best = {name: float("inf") for name in trainers}
    for _ in range(repeats):
        for name, fn in trainers.items():
            best[name] = min(best[name], fn(epochs) / epochs * 1e3)
    return best


timers = {"fixed": lambda n: train_strong(epochs=n, log_every=QUIET)[2],
          "grad. balance": lambda n: train_adaptive("gradbal", epochs=n,
log_every=QUIET)[2],
          "Wang, capped": lambda n: train_adaptive("wang",
alpha_max=ALPHA_MAX, epochs=n, log_every=QUIET)[2]}
cost = ms_per_epoch(timers)
print("\nCost per epoch (best of 3 x 500 epochs):")
for name, ms in cost.items():
    print(f"  {name:14s} {ms:6.2f} ms  ({ms / cost['fixed']:.2f}x fixed)")
```

```
[strong] epoch      1 | L=1.914e+02 | L_BC=7.770e-02 | L_phys=1.913e+02
[gradbal] epoch      1 | L_BC=7.770e-02 | L_phys=1.913e+02 | alpha=1.09 |
beta=0.907
[wang] epoch      1 | L_BC=7.770e-02 | L_phys=1.913e+02 | alpha=66.7 | beta=1
[wang, cap 1000] epoch      1 | L_BC=7.770e-02 | L_phys=1.913e+02 |
alpha=66.7 | beta=1
[strong] epoch      1 | L=2.005e+02 | L_BC=4.745e-02 | L_phys=2.005e+02
[gradbal] epoch      1 | L_BC=4.745e-02 | L_phys=2.005e+02 | alpha=1.09 |
beta=0.905
[wang] epoch      1 | L_BC=4.745e-02 | L_phys=2.005e+02 | alpha=57.1 | beta=1
[wang, cap 1000] epoch      1 | L_BC=4.745e-02 | L_phys=2.005e+02 |
alpha=57.1 | beta=1

scheme         seed  final rel. L2  best rel. L2  stalled epochs
fixed             0      1.047e-03     1.544e-06               0
fixed             1      3.016e-03     5.648e-06               0
```

```
fixed             2      7.246e-05     4.629e-06              0
grad. balance     0      3.641e-05     3.540e-05           4346
grad. balance     1      8.390e-06     8.346e-06           3318
grad. balance     2      4.730e-03     1.629e-04           3858
Wang              0      9.988e-01     9.492e-01           9500
Wang              1      1.001e+00     9.235e-01           9500
Wang              2      9.972e-01     9.667e-01           9500
Wang, capped      0      2.943e-02     7.920e-05              0
Wang, capped      1      4.884e-04     2.785e-04              0
Wang, capped      2      5.615e-05     1.777e-05              0

Cap sensitivity (seed 0):
[wang, cap 100] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 | alpha=10.9
| beta=1
  alpha_max =   1e+02: final rel. L2 = 3.451e-05, best rel. L2 = 5.173e-05
  alpha_max =   1e+03: final rel. L2 = 2.943e-02, best rel. L2 = 7.920e-05
[wang, cap 10000] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 |
alpha=345 | beta=1
  alpha_max =   1e+04: final rel. L2 = 6.349e-04, best rel. L2 = 4.236e-05
[strong] epoch      1 | L=1.978e+02 | L_BC=2.655e-03 | L_phys=1.978e+02
[gradbal] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 | alpha=1.1 |
beta=0.901
[wang, cap 1000] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 | alpha=101
| beta=1
[strong] epoch      1 | L=1.978e+02 | L_BC=2.655e-03 | L_phys=1.978e+02
[gradbal] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 | alpha=1.1 |
beta=0.901
[wang, cap 1000] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 | alpha=101
| beta=1
[strong] epoch      1 | L=1.978e+02 | L_BC=2.655e-03 | L_phys=1.978e+02
[gradbal] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 | alpha=1.1 |
beta=0.901
[wang, cap 1000] epoch      1 | L_BC=2.655e-03 | L_phys=1.978e+02 | alpha=101
| beta=1

Cost per epoch (best of 3 x 500 epochs):
  fixed            1.63 ms  (1.00x fixed)
  grad. balance    1.91 ms  (1.17x fixed)
  Wang, capped     1.85 ms  (1.13x fixed)
```

Discussion.

Gradient-norm balancing. α reached its ceiling of 2 within the first thousand epochs, while β fell to between 10^{-7} and 10^{-3}. The boundary loss dropped to about 10^{-13}, compared with 6.8×10^{-6} for the fixed run. Because $\alpha + \beta = 2$ and $\alpha g_b = \beta g_p$, the weighted gradient is about $2g_b$ once $g_b \ll g_p$, and g_b is tiny once the boundary conditions are fitted. Adam's second-moment estimate still remembers the earlier large gradients, so these small gradients produce very small steps. $\mathcal{L}_{Physics}$ therefore stalled at 0.94 between epochs 1,000 and 4,000 before falling to 5.8×10^{-4}. The stall happens on every seed tested: $\mathcal{L}_{Physics}$ stayed above 0.5 for 4,346, 3,318 and 3,858 epochs on seeds 0, 1 and 2.

Wang annealing, uncapped. This scheme failed on every seed, with final relative L^2 errors of 0.997 to 1.001 (0.92 to 0.97 even at the best checkpoint). The cause is a feedback loop in the formula. The denominator is the mean BC gradient, and with only two boundary points the network can satisfy the BCs almost exactly, so the denominator goes to zero and $\hat{\alpha}$ grows. A larger α makes the BCs fit even better, which shrinks the denominator further, and α reached 1.1×10^{10}. Since only α/β matters under Adam, the physics gradient was effectively invisible at that ratio. $\mathcal{L}_{Physics}$ stayed at about 196 (it started at 198), and the network learned $T \approx 0$, which satisfies the boundary conditions and ignores the PDE.

Wang annealing, capped at $\alpha_{max} = 10^3$. The cap breaks the loop. α stayed mostly below the cap (between about 200 and 550 at the logged epochs), $\mathcal{L}_{Physics}$ fell steadily, and the lowest-loss checkpoint reached 7.9×10^{-5}, with 1.8×10^{-5} to 2.8×10^{-4} across seeds. Caps of 10^2 and 10^4 gave similar checkpoints (5.2×10^{-5} and 4.2×10^{-5}), so the exact value is not critical. Its seed-0 final epoch landed on an Adam spike, with an error of 2.9×10^{-2}.

Is the final-epoch improvement real? At the final epoch, gradient balancing looks 29 times better than the fixed run (3.6×10^{-5} against 1.05×10^{-3}). That margin mostly reflects where the last Adam step happened to land, because the fixed run's final epoch fell on a spike (Problem 1). Comparing the lowest-loss checkpoints reverses the ranking. The fixed run reaches 1.5×10^{-6}, 5.6×10^{-6} and 4.6×10^{-6} on the three seeds, better than every adaptive run: gradient balancing gives 8.3×10^{-6} to 1.6×10^{-4} and capped Wang 1.8×10^{-5} to 2.8×10^{-4}. Final-epoch errors vary by up to nearly three orders of magnitude between seeds for the same scheme, so they cannot rank the methods.

Conclusion. On this problem, adaptive weighting did not improve accuracy. The gradient imbalance it targets is real at initialization, but two boundary points are easy to satisfy, and the fixed run's boundary loss already drops to about 10^{-10} (epoch 9,000). Once the BCs are fitted, upweighting them either starves the physics term (gradient balancing) or, without a cap, overwhelms it (Wang). Adaptive weights are more likely to pay off when the boundary or data terms are genuinely hard to fit.

Cost. The adaptive schemes are slightly more expensive per epoch. In interleaved short runs, gradient balancing cost 1.17 times the fixed run and capped Wang 1.13 times (1.91 and 1.85 ms against 1.63 ms per epoch). The overhead comes from taking the two loss gradients separately and computing the weights. No extra forward pass is needed. Full-run wall-clock times (18 to 21 s) vary by up to a factor of two between runs on this laptop, so they are not a reliable basis for comparing cost.

Problem 3: Variational Formulation

Part a: Weak form derivation

Write the PDE as $T'' + f = 0$ with $f = q/\kappa$. Multiply by a test function $\phi(x)$ with $\phi(0) = \phi(1) = 0$ and integrate:

$$\int_0^1 T''\phi \, dx + \int_0^1 f\phi \, dx = 0$$

Integrating the first term by parts,

$$\int_0^1 T''\phi \, dx = \left[T'\phi\right]_0^1 - \int_0^1 T'\phi' \, dx$$

The boundary term vanishes because ϕ is zero at both ends, which leaves the weak form

$$\boxed{\int_0^1 \frac{dT}{dx}\frac{d\phi}{dx} \, dx = \int_0^1 \frac{q(x)}{\kappa}\,\phi \, dx \quad \text{for every admissible } \phi}$$

Only the first derivative of T is needed, so the weak form asks less smoothness of the solution and needs one fewer autograd pass than the strong form.

Part b: Implementation

Test functions. $K = 20$ sine modes, $\phi_k(x) = \sqrt{2}\,\sin(k\pi x)/(k\pi)$, $k = 1, \ldots, 20$. Each vanishes at both boundaries. The $1/(k\pi)$ factor normalizes $\|\phi_k'\|_{L^2} = 1$, so every residual has a comparable scale and the high-frequency modes do not dominate the loss.

Quadrature. 100-point Gauss-Legendre quadrature mapped to $[0, 1]$, with nodes x_j and weights w_j. Using 100 points keeps the number of network evaluations equal to the strong form.

Loss.

$$R_k = \sum_j w_j\, T_\theta'(x_j)\, \phi_k'(x_j) - \sum_j w_j\, f(x_j)\, \phi_k(x_j), \qquad \mathcal{L} = \mathcal{L}_{BCs} + \frac{1}{K}\sum_{k=1}^{K} R_k^2$$

The Dirichlet conditions are still imposed through $\mathcal{L}_{BCs}$, so the only difference from Problem 1 is the PDE term. Everything that does not depend on the network is precomputed once, so each epoch costs one forward pass, one first derivative and one matrix-vector product. Network, initialization, optimizer and epochs are identical to Problem 1.

This is the plain VPINN formulation: global sine test functions on a single element. An hp-VPINN would instead split the domain into elements with local test functions.

In [10]:
```
def build_weak_form(n_quad=100, n_test=20):
    xi, wi = np.polynomial.legendre.leggauss(n_quad)      # nodes/weights
on [-1, 1]
    xq, wq = 0.5 * (xi + 1.0), 0.5 * wi                   # map to [0, 1]
    kp = np.arange(1, n_test + 1) * np.pi
    phi = np.sqrt(2) * np.sin(np.outer(xq, kp)) / kp      # phi_k(x_j),
(N_q, K)
    dphi = np.sqrt(2) * np.cos(np.outer(xq, kp))          # phi_k'(x_j),
(N_q, K)
    F = (wq * q(xq) / KAPPA) @ phi                        # right-hand
side, (K,)
    to_t = lambda a: torch.tensor(a, device=DEVICE,
dtype=torch.get_default_dtype())
    x_q = to_t(xq).reshape(-1, 1).requires_grad_(True)
    A = to_t(wq[:, None] * dphi)                          # quadrature
weights folded into phi'
    return x_q, A, to_t(F)


def weak_loss(model, x_q, A, F):
    T_x = grad(model(x_q), x_q)                           # only the
first derivative
    R = (T_x * A).sum(dim=0) - F                          # weak residual
per test function
    return torch.mean(R**2)


def train_weak(epochs=10_000, lr=1e-3, n_quad=100, n_test=20, seed=0,
log_every=1000):
    set_seed(seed)
    model = make_model()
    x_q, A, F = build_weak_form(n_quad, n_test)
    x_b = boundary_points()
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    hist = {"total": [], "bc": [], "phys": []}
    best = BestCheckpoint()
    t0 = time.perf_counter()
    for epoch in range(1, epochs + 1):
        opt.zero_grad(set_to_none=True)
        l_bc, l_w = bc_loss(model, x_b), weak_loss(model, x_q, A, F)
        loss = l_bc + l_w
        best.update(epoch, loss.item(), model)
        loss.backward()
        opt.step()
        hist["total"].append(loss.item()); hist["bc"].append(l_bc.item());
hist["phys"].append(l_w.item())
        if epoch % log_every == 0 or epoch == 1:
            print(f"[weak] epoch {epoch:6d} | L={loss.item():.3e} | L_BC=
{l_bc.item():.3e} | L_weak={l_w.item():.3e}")
    best.store(hist)
    return model, hist, time.perf_counter() - t0
```

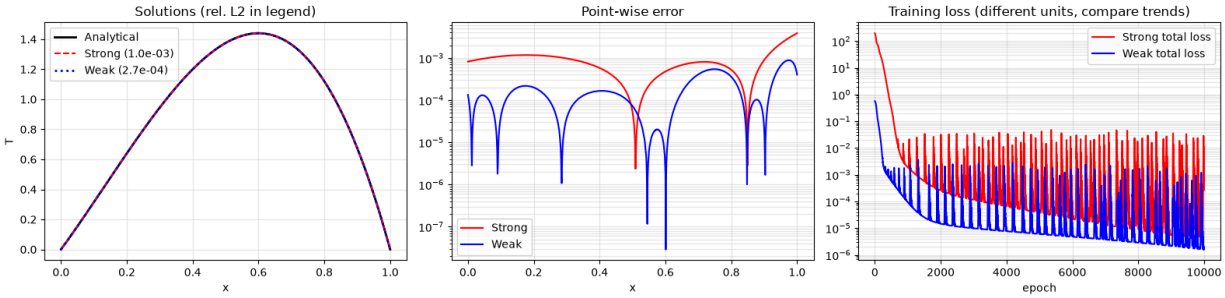
In [11]:
```
m_weak, h_weak, t_weak = train_weak()
_, T_w, _, err_weak = evaluate(m_weak)
```

```
print(f"\nstrong form: rel. L2 = {err_strong:.3e},  time = {t_strong:.1f}
s")
print(f"weak form  : rel. L2 = {err_weak:.3e},  time = {t_weak:.1f} s")
err_weak_best = evaluate(best_model(h_weak))[3]
print(f"lowest-loss checkpoints: strong {err_strong_best:.3e} (epoch
{h_strong['best_epoch']}), "
      f"weak {err_weak_best:.3e} (epoch {h_weak['best_epoch']})")

fig, ax = plt.subplots(1, 3, figsize=(16, 4))
ax[0].plot(x_test, T_true, "k-", lw=2, label="Analytical")
ax[0].plot(x_test, T_s, "r--", label=f"Strong ({err_strong:.1e})")
ax[0].plot(x_test, T_w, "b:", lw=2, label=f"Weak ({err_weak:.1e})")
ax[0].set_title("Solutions (rel. L2 in legend)"); ax[0].set_xlabel("x");
ax[0].set_ylabel("T")
ax[1].semilogy(x_test, np.abs(T_s - T_true) + 1e-16, "r", label="Strong")
ax[1].semilogy(x_test, np.abs(T_w - T_true) + 1e-16, "b", label="Weak")
ax[1].set_title("Point-wise error"); ax[1].set_xlabel("x")
ax[2].semilogy(h_strong["total"], "r", label="Strong total loss")
ax[2].semilogy(h_weak["total"], "b", label="Weak total loss")
ax[2].set_title("Training loss (different units, compare trends)");
ax[2].set_xlabel("epoch")
for a in ax: a.legend(); a.grid(alpha=0.3, which="both")
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/p3_strong_vs_weak.png",
dpi=150); plt.show()
```

```
[weak] epoch      1 | L=5.685e-01 | L_BC=2.655e-03 | L_weak=5.659e-01
[weak] epoch   1000 | L=9.379e-05 | L_BC=4.880e-08 | L_weak=9.374e-05
[weak] epoch   2000 | L=1.479e-05 | L_BC=2.755e-08 | L_weak=1.476e-05
[weak] epoch   3000 | L=9.927e-06 | L_BC=2.173e-08 | L_weak=9.905e-06
[weak] epoch   4000 | L=8.174e-06 | L_BC=4.306e-07 | L_weak=7.744e-06
[weak] epoch   5000 | L=9.293e-06 | L_BC=3.165e-06 | L_weak=6.128e-06
[weak] epoch   6000 | L=9.082e-06 | L_BC=4.256e-06 | L_weak=4.825e-06
[weak] epoch   7000 | L=7.814e-05 | L_BC=7.367e-05 | L_weak=4.473e-06
[weak] epoch   8000 | L=2.879e-06 | L_BC=7.373e-10 | L_weak=2.879e-06
[weak] epoch   9000 | L=2.176e-06 | L_BC=1.752e-09 | L_weak=2.174e-06
[weak] epoch  10000 | L=1.621e-06 | L_BC=3.870e-08 | L_weak=1.583e-06

strong form: rel. L2 = 1.047e-03,  time = 18.2 s
weak form  : rel. L2 = 2.706e-04,  time = 11.5 s
lowest-loss checkpoints: strong 1.544e-06 (epoch 9929), weak 2.224e-04 (epoch
9999)
```



In [12]:
```
weak_seed_err = {0: (err_weak, err_weak_best)}
for s in SEEDS[1:]:
    m, h, _ = train_weak(seed=s, log_every=QUIET)
    weak_seed_err[s] = (evaluate(m)[3], evaluate(best_model(h))[3])

print(f"\n{'seed':>4s} {'strong final':>13s} {'weak final':>11s} {'strong
```

```
best':>12s} {'weak best':>10s}")
for s in SEEDS:
    sf, sb = seed_err["fixed"][s][:2]
    wf, wb = weak_seed_err[s]
    print(f"{s:4d} {sf:13.3e} {wf:11.3e} {sb:12.3e} {wb:10.3e}")

cost_p3 = ms_per_epoch({"strong": lambda n: train_strong(epochs=n,
log_every=QUIET)[2],
                        "weak": lambda n: train_weak(epochs=n,
log_every=QUIET)[2]})
print(f"\nCost per epoch: strong {cost_p3['strong']:.2f} ms, weak
{cost_p3['weak']:.2f} ms "
      f"({cost_p3['weak'] / cost_p3['strong']:.2f}x)")
```

```
[weak] epoch      1 | L=6.219e-01 | L_BC=7.770e-02 | L_weak=5.442e-01
[weak] epoch      1 | L=6.244e-01 | L_BC=4.745e-02 | L_weak=5.769e-01

seed  strong final  weak final  strong best  weak best
   0     1.047e-03   2.706e-04    1.544e-06  2.224e-04
   1     3.016e-03   1.304e-04    5.648e-06  1.321e-04
   2     7.246e-05   3.019e-04    4.629e-06  1.139e-04
[strong] epoch      1 | L=1.978e+02 | L_BC=2.655e-03 | L_phys=1.978e+02
[weak] epoch      1 | L=5.685e-01 | L_BC=2.655e-03 | L_weak=5.659e-01
[strong] epoch      1 | L=1.978e+02 | L_BC=2.655e-03 | L_phys=1.978e+02
[weak] epoch      1 | L=5.685e-01 | L_BC=2.655e-03 | L_weak=5.659e-01
[strong] epoch      1 | L=1.978e+02 | L_BC=2.655e-03 | L_phys=1.978e+02
[weak] epoch      1 | L=5.685e-01 | L_BC=2.655e-03 | L_weak=5.659e-01

Cost per epoch: strong 3.30 ms, weak 2.58 ms (0.78x)
```

Discussion. At the final epoch, the weak form reached a relative L^2 error of 2.7×10^{-4}, 3.9 times lower than the strong form (1.05×10^{-3}). That comparison is unreliable for the reason found in Problems 1 and 2: the strong form's final epoch landed on an Adam spike. Across three seeds the final-epoch ranking is mixed (weak is better on seeds 0 and 1, strong on seed 2). Comparing the lowest-training-loss checkpoints reverses it. The strong form reaches 1.5×10^{-6} to 5.6×10^{-6} against 1.1×10^{-4} to 2.2×10^{-4} for the weak form, which is 23 to 144 times better for the strong form.

What the weak form does deliver is steadier training. Its final-epoch error stays between 1.3×10^{-4} and 3.0×10^{-4} across seeds, while the strong form ranges from 7.2×10^{-5} to 3.0×10^{-3}, and the gap between its final and best errors is small. Integrating against smooth test functions averages the residual over the domain instead of enforcing it point by point, and working with T' instead of T'' lowers the derivative order, so a smoother loss surface is a plausible explanation. The lower accuracy ceiling also has a plausible cause, although it was not tested here. The weak residual only sees the projection of the error onto the 20 test functions, and the weak loss starts about 350 times smaller than the strong loss (0.57 against 198), so its gradient signal near convergence is weaker. The weak form is cheaper because it needs one fewer autograd pass. Per epoch it cost 0.78 times the strong form in interleaved short runs (2.58 against 3.30 ms), and the full runs took 11.5 s against 18.2 s.

Limitations. The two loss values have different units and should be compared by trend only. Three seeds give an indication of the spread but not a precise estimate. The weak form also only constrains the solution within the span of the 20 test functions. That is sufficient here because the exact solution is a smooth cubic, but problems with sharper features would need more modes or local test functions (hp-VPINN).

Problem 4: Data-Driven Cross-Section Identification

$$\frac{d}{dx}\left(EA(x)\frac{du}{dx}\right) + p(x) = 0, \quad x \in [0,1], \qquad u(x) = \sin(2\pi x)$$

$$p(x) = -2(3x^2 - 2x)\pi\cos(2\pi x) + 4(x^3 - x^2 + 1)\pi^2 \sin(2\pi x)$$

Consistency check of the load. With $EA = x^3 - x^2 + 1$, $EA' = 3x^2 - 2x$, $u' = 2\pi\cos(2\pi x)$ and $u'' = -4\pi^2\sin(2\pi x)$:

$$(EA\,u')' = EA'u' + EA\,u'' = 2\pi(3x^2 - 2x)\cos(2\pi x) - 4\pi^2(x^3 - x^2 + 1)\sin(2\pi x) = -p(x)$$

so the given load is exactly consistent with the target stiffness.

Part a: Network design

$EA_\theta(x, u)$ has two inputs (position and displacement), three hidden layers of 20 tanh neurons and a linear output. Unlike Problems 1 to 3, the network does not predict the PDE solution. It predicts a coefficient of the PDE, and the known displacement enters the residual.

Part b: Physics-informed loss

$$r(x_i) = \frac{d}{dx}\Big[EA_\theta\big(x_i, u(x_i)\big)\,u'(x_i)\Big] + p(x_i), \qquad \mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} r(x_i)^2$$

Because EA_θ depends on x both directly and through u, the total derivative is $\frac{dEA}{dx} = \frac{\partial EA}{\partial x} + \frac{\partial EA}{\partial u}u'$. Autograd applies this chain rule automatically: u is computed from x inside the graph, the axial force $N = EA\,u'$ is formed, and dN/dx is taken with respect to x.

Identifiability. The residual fixes the axial force $EA\,u'$ only up to an additive constant. That constant is pinned at the points where $u' = 0$ ($x = 0.25$ and $x = 0.75$): a smooth network cannot represent $EA = (C - P(x))/u'$ unless C takes the correct value,

because EA would otherwise diverge there. EA is therefore identifiable from the residual alone, but it is least constrained near those two points.

In [13]:
```
PI = np.pi

def u_exact(x):
    return torch.sin(2 * PI * x)

def p_load(x):
    return (-2 * (3 * x**2 - 2 * x) * PI * torch.cos(2 * PI * x)
            + 4 * (x**3 - x**2 + 1) * PI**2 * torch.sin(2 * PI * x))

def EA_exact(x):
    return x**3 - x**2 + 1


def residual_loss(ea_net, x, u_fn):
    u = u_fn(x)
    u_x = grad(u, x)                                    # du/dx
    EA = ea_net(torch.cat([x, u], dim=1))               # input pair (x, u)
    flux_x = grad(EA * u_x, x)                          # d/dx(EA u'), chain
rule via autograd
    return torch.mean((flux_x + p_load(x))**2)


def train_EA(u_fn, epochs=5000, lr=1e-3, n=100, seed=0, tag="clean",
log_every=1000):
    set_seed(seed)
    ea_net = MLP(in_dim=2, out_dim=1, hidden=20, n_layers=3).to(DEVICE)
    x = torch.linspace(0, 1, n, device=DEVICE).reshape(-1,
1).requires_grad_(True)
    opt = torch.optim.Adam(ea_net.parameters(), lr=lr)
    hist = []
    for epoch in range(1, epochs + 1):
        opt.zero_grad(set_to_none=True)
        loss = residual_loss(ea_net, x, u_fn)
        loss.backward()
        opt.step()
        hist.append(loss.item())
        if epoch % log_every == 0 or epoch == 1:
            print(f"[EA-{tag}] epoch {epoch:5d} | L_phys=
{loss.item():.3e}")
    return ea_net, x, hist


def predict_EA(ea_net, x, u_fn):
    with torch.no_grad():
        return to_numpy(ea_net(torch.cat([x, u_fn(x)], dim=1)))


print("EA network parameters:", n_params(MLP(2, 1, 20, 3)))
```

```
EA network parameters: 921
```

Parts c and d: Training and comparison with the analytical solution

$N = 100$ uniformly spaced points, Adam with learning rate 10^{-3}, 5,000 full-batch epochs.

In [14]:
```
N, SIGMA = 100, 0.01
x_np = np.linspace(0, 1, N)
EA_true = EA_exact(x_np)
ea_results = {}

net, x_t, h = train_EA(u_exact, tag="clean")
ea_results["clean"] = (predict_EA(net, x_t, u_exact), h)
print(f"\nClean: relative L2 error of EA = {relative_l2(ea_results['clean']
[0], EA_true):.3e}")

e_pw = np.abs(ea_results["clean"][0] - EA_true)
peaks = [i for i in range(1, N - 1) if e_pw[i] >= e_pw[i - 1] and e_pw[i]
>= e_pw[i + 1]]
top = sorted(sorted(peaks, key=lambda i: -e_pw[i])[:3])
print("Largest local error peaks: " + ", ".join(f"x = {x_np[i]:.3f}
({e_pw[i]:.1e})" for i in top))
```

```
[EA-clean] epoch     1 | L_phys=8.395e+02
[EA-clean] epoch  1000 | L_phys=7.641e-02
[EA-clean] epoch  2000 | L_phys=2.002e-02
[EA-clean] epoch  3000 | L_phys=1.126e-02
[EA-clean] epoch  4000 | L_phys=6.103e-03
[EA-clean] epoch  5000 | L_phys=3.069e-03

Clean: relative L2 error of EA = 7.806e-04
Largest local error peaks: x = 0.152 (8.3e-04), x = 0.293 (1.5e-03), x =
0.758 (1.5e-03)
```

Discussion. The identified stiffness has a relative L^2 error of 7.8×10^{-4}. The point-wise error (figure below) stays under about 1.5×10^{-3}. Its two largest peaks, both about 1.5×10^{-3}, are at $x \approx 0.29$ and $x \approx 0.76$, just outside the points where $u' = 0$ (0.25 and 0.75), with a smaller third peak at $x \approx 0.15$. The 0.76 peak sits almost on its $u' = 0$ point, but the 0.29 peak is offset by about 0.04, so this is loose support for the identifiability argument rather than a precise confirmation. The residual loss was still decreasing at epoch 5,000, so more epochs would likely improve accuracy further.

Part e: Influence of measurement noise

Noisy measurements: $u_{obs}(x_i) = \sin(2\pi x_i) + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, 0.01^2)$. Noisy data exist only at discrete points, so u' and u'' cannot be obtained from them by autograd. Two treatments are compared.

- **Naive input noise.** Noise perturbs only the displacement fed into the network, while the derivatives still come from the exact $\sin(2\pi x)$. This isolates the effect of noise on the network input and is optimistic, because the derivatives in the residual stay noise-free.
- **Surrogate fit (realistic).** A separate tanh network $u_\phi(x)$ ($1 \to 20 \to 20 \to 20 \to 1$) is first fitted to the noisy data, with $u(0) = u(1) = 0$ added as a penalty, for 5,000 Adam epochs. It is then frozen, and u_ϕ with its autograd derivatives replaces the exact displacement in the residual. Noise now reaches u' and u'', which is where differentiation amplifies it.

In [15]:
```
def fit_u_surrogate(x_np, u_obs_np, epochs=5000, lr=1e-3, seed=0):
    set_seed(seed)
    u_net = MLP(1, 1, 20, 3).to(DEVICE)
    to_t = lambda a: torch.tensor(a, device=DEVICE,
dtype=torch.get_default_dtype()).reshape(-1, 1)
    x_d, u_d = to_t(x_np), to_t(u_obs_np)
    x_b = torch.tensor([[0.0], [1.0]], device=DEVICE)
    opt = torch.optim.Adam(u_net.parameters(), lr=lr)
    for _ in range(epochs):
        opt.zero_grad(set_to_none=True)
        loss = torch.mean((u_net(x_d) - u_d)**2) +
torch.mean(u_net(x_b)**2)
        loss.backward()
        opt.step()
    print(f"[u-surrogate] final data MSE = {loss.item():.3e}  (noise
variance = {SIGMA**2:.1e})")
    for p in u_net.parameters():
        p.requires_grad_(False)                          # freeze before
training the EA network
    return u_net


rng = np.random.default_rng(1)
noise_np = SIGMA * rng.standard_normal(N)
u_obs_np = np.sin(2 * PI * x_np) + noise_np
noise_t = torch.tensor(noise_np, device=DEVICE,
dtype=torch.get_default_dtype()).reshape(-1, 1)

# Naive: noise_t is constant with respect to x, so it changes the input but
not u'
u_naive = lambda x: u_exact(x) + noise_t
net, x_t, h = train_EA(u_naive, tag="naive")
ea_results["noisy (naive)"] = (predict_EA(net, x_t, u_naive), h)

# Surrogate: derivatives come from a network fitted to the noisy data
u_net = fit_u_surrogate(x_np, u_obs_np)
net, x_t, h = train_EA(u_net, tag="surrogate")
ea_results["noisy (surrogate)"] = (predict_EA(net, x_t, u_net), h)

err_clean = relative_l2(ea_results["clean"][0], EA_true)
print(f"\n{'case':20s} {'rel. L2':>10s} {'final loss':>11s} {'vs
clean':>9s}")
for name, (EA_pred, h) in ea_results.items():
```

```
    e = relative_l2(EA_pred, EA_true)
    print(f"{name:20s} {e:10.3e} {h[-1]:11.3e} {e / err_clean:8.1f}x")
```

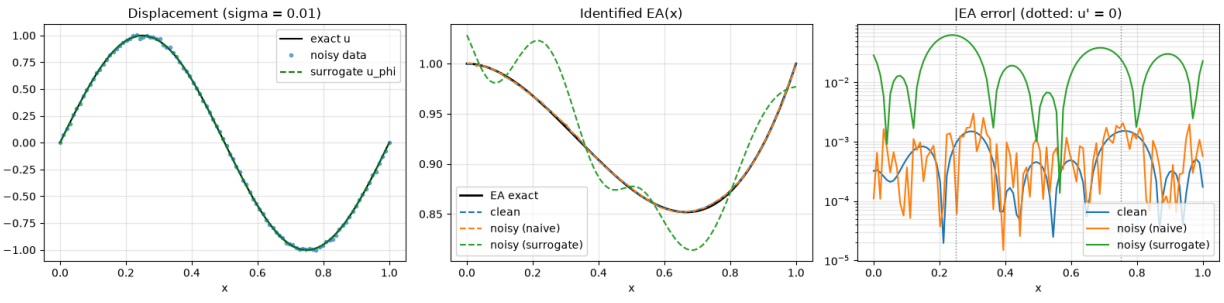
```
[EA-naive] epoch     1 | L_phys=8.402e+02
[EA-naive] epoch  1000 | L_phys=7.756e-02
[EA-naive] epoch  2000 | L_phys=2.070e-02
[EA-naive] epoch  3000 | L_phys=1.177e-02
[EA-naive] epoch  4000 | L_phys=6.516e-03
[EA-naive] epoch  5000 | L_phys=3.441e-03
[u-surrogate] final data MSE = 7.873e-05  (noise variance = 1.0e-04)
[EA-surrogate] epoch     1 | L_phys=8.422e+02
[EA-surrogate] epoch  1000 | L_phys=5.431e-01
[EA-surrogate] epoch  2000 | L_phys=2.193e-01
[EA-surrogate] epoch  3000 | L_phys=9.064e-02
[EA-surrogate] epoch  4000 | L_phys=5.212e-02
[EA-surrogate] epoch  5000 | L_phys=3.410e-02

case                     rel. L2  final loss  vs clean
clean                  7.806e-04   3.069e-03      1.0x
noisy (naive)          1.030e-03   3.441e-03      1.3x
noisy (surrogate)      2.973e-02   3.410e-02     38.1x
```

In [16]:

```
with torch.no_grad():
    u_sur_np = to_numpy(u_net(torch.tensor(x_np,
dtype=torch.get_default_dtype()).reshape(-1, 1)))

fig, ax = plt.subplots(1, 3, figsize=(16, 4))
ax[0].plot(x_np, np.sin(2 * PI * x_np), "k-", label="exact u")
ax[0].plot(x_np, u_obs_np, "o", ms=3, alpha=0.6, label="noisy data")
ax[0].plot(x_np, u_sur_np, "g--", label="surrogate u_phi")
ax[0].set_title(f"Displacement (sigma = {SIGMA})")
ax[1].plot(x_np, EA_true, "k-", lw=2, label="EA exact")
for name, (EA_pred, _) in ea_results.items():
    ax[1].plot(x_np, EA_pred, "--", label=name)
    ax[2].semilogy(x_np, np.abs(EA_pred - EA_true) + 1e-16, label=name)
for xv in (0.25, 0.75):
    ax[2].axvline(xv, color="gray", ls=":", lw=1)
ax[1].set_title("Identified EA(x)"); ax[2].set_title("|EA error| (dotted:
u' = 0)")
for a in ax: a.set_xlabel("x"); a.legend(); a.grid(alpha=0.3, which="both")
fig.tight_layout(); fig.savefig(f"{FIG_DIR}/p4_EA_identification.png",
dpi=150); plt.show()
```



Discussion. Noise that only perturbs the network input has a small effect: the error rises from 7.8×10^{-4} to 1.0×10^{-3} (1.3 times), and the point-wise error becomes jagged because each input point carries a different random offset, but the identified curve is

visually indistinguishable from the exact stiffness. The network relies mainly on the x input and tolerates small perturbations in u.

The realistic case is far more sensitive. The surrogate reproduces the displacement well at plot scale, yet the identified EA oscillates around the true curve with errors of up to about 6×10^{-2} near $x \approx 0.22$, and the relative error is 3.0×10^{-2}, 38 times the clean value. The residual depends on u' and u'', and differentiating a fit amplifies any small wiggles it contains. The surrogate's final data misfit (MSE 7.9×10^{-5}) is below the noise variance (1.0×10^{-4}), which indicates it has partly fitted the noise itself. Those wiggles appear in u'' and propagate into the stiffness.

Ways to reduce this sensitivity include stopping the surrogate fit once the data misfit reaches the noise variance, using a smaller surrogate or weight decay, training the surrogate and EA network jointly so the physics constrains the displacement fit, or using a smoothing spline for the displacement. Averaging the identified EA over several noise realizations would also separate random from systematic error.